Session 6A: HTTP, FastAPI, and Pydantic

Trainer Reference Guide

Purpose and outcomes

Explain client/server exchange, routes, HTTP methods/status codes, JSON, request schemas, and automatic API documentation.

Examples and exercises

- 01_http_json_api.ipynb maps actions to GET/POST/PUT/DELETE and status codes 200, 201, 204, 404.
- 02_fastapi_basics.py exposes root/list/create/detail routes over an in-memory list and returns 404 for missing users.
- 03_pydantic_models.py validates required text, EmailStr, and an age range.
- exercise_01_routes.py is guided health/departments routing. exercise_02_schema.py independently constrains a ticket. challenge_user_api.py completes in-memory CRUD.

Trainer key points specific to this module

Use Swagger '/docs' as an interactive client and inspect request/response bodies. Distinguish route parameters, query parameters, forms, and JSON. Explain 422 as useful validation feedback and 404 as a resource result, not a server crash.

Portal connection and evidence

Learners can create valid JSON, interpret rejected input, and select correct methods/statuses before adding persistence.

Delivery pattern

1. Explain the idea briefly and ask learners to predict a result.
2. Run the simple example, then connect it to an employee/user workflow.
3. Pair learners for the guided task; require an individual attempt next.
4. Use the challenge as the cumulative project increment.
5. Review the quiz and ask learners to explain, not only show, their code.

Trainer key points

- Keep live coding visible and deliberate. Normalize reading tracebacks.
- Ask what type and value each expression produces before running it.
- Let TODO files remain student-owned; open solutions only during review.
- Check boundary and failure cases, not only the happy path.
- End by tracing how the topic appears in the final portal.